

RESUMO COMPLETO

Programação Orientada a Objetos em Java

com base na Prova do Prof. Rodrigo Pagliares — UNIFAL-MG

Unidades I e II — Ambiente, OO, Herança, Padrões e Exceções

1. Padrão Singleton

O padrão Singleton garante que uma classe tenha apenas uma instância e fornece um ponto global de acesso a ela. É cobrado diretamente na prova (Questão 1).

1.1 Estrutura obrigatória

- Construtor privado — impede new fora da classe
- Atributo estático final do próprio tipo — guarda a única instância
- Método público estático de acesso — retorna a instância

```
public class Albergue {  
    private String nome;  
    // 1. Instância única criada na carga da classe  
    private static final Albergue albergue = new Albergue();  
    // 2. Construtor privado — bloqueia new externo  
    private Albergue() { }  
    // 3. Acesso global estático  
    public static Albergue getAlbergue() {  
        return albergue;  
    }  
}
```

Ponto de atenção da prova

O método `getAlbergue()` DEVE ser `static` (não existe objeto para chamá-lo antes da primeira chamada) e DEVE retornar o tipo `Albergue`.

2. Classes, Objetos e Variáveis

Coberto nas Questões 2, 3, 4 e 5. Fundamento de toda a prova.

2.1 Variáveis de instância vs de classe (`static`)

Variável de instância	Variável de classe (<code>static</code>)
-----------------------	--

Uma cópia por objeto criado	Uma única cópia, compartilhada por todos
Declarada sem static	Declarada com static
Ex.: <code>private String nome;</code>	Ex.: <code>private static int contador;</code>
Acesso via referência de objeto	Acesso via nome da classe ou instância

```

public class Albergue {
private Reserva[] reservas = new Reserva[3]; // instância (2.1)
private static int contador; // classe (2.2)
public boolean adicionarReserva(String dataReserva) {
Reserva reserva = new Reserva(); // (2.3)
reserva.setDataReserva(dataReserva); // (2.4)
reservas[contador] = reserva; // (2.5a)
contador++; // (2.5b)
return true;
}
}

```

Por que contador é static?

O contador controla o próximo índice livre no array. Ele pertence ao estado da classe (não de um objeto específico) — por isso é static. Se fosse de instância, cada objeto teria seu próprio contador zerado.

3. Construtores e Sobrecarga com this()

Questão 3 da prova. Sobrecarga de construtores + delegação interna com this().

3.1 Regras dos construtores

- Mesmo nome da classe, sem tipo de retorno
- Se nenhum for declarado, o compilador gera um padrão (sem args)
- Se qualquer construtor for declarado, o padrão deixa de existir automaticamente
- `this(...)` chama outro construtor da mesma classe — DEVE ser a primeira instrução
- `super(...)` chama o construtor da superclasse — também DEVE ser a primeira instrução

```

public class Hospede {
    private String nome;
    private String sobrenome;
    // Construtor padrão: delega para o de 2 parâmetros
    public Hospede() {
        this("", ""); // 3.1 — deve ser a 1ª linha
    }
    // Construtor com 1 parâmetro: delega para o de 2
    public Hospede(String nome) {
        this(nome, ""); // 3.1
    }
    // Construtor principal: inicializa diretamente
    public Hospede(String nome, String sobrenome) {
        this.nome = nome; // 3.2 — resolve ambiguidade
        this.sobrenome = sobrenome; // 3.2
    }
}

```

3.2 O operador this

Uso de this	Finalidade
this.nome = nome;	Resolve ambiguidade entre atributo e parâmetro de mesmo nome
this("arg");	Chama outro construtor da mesma classe (delegação)
return this;	Retorna o próprio objeto (padrão Builder/fluent API)

4. Modificadores de Acesso e Herança

Questões 4 e 5 exploram modificadores em contexto de herança.

Modificador	Acesso na mesma classe	Acesso em subclasse / fora
private	Sim	Não — nem em subclasses
default (sem mod.)	Sim	So no mesmo pacote

protected	Sim	Subclasses de qualquer pacote + mesmo pacote
public	Sim	Qualquer lugar

protected na prova (4.2)

O atributo valorHospedagem da classe Pagamento recebe protected para que as subclasses (PagamentoDinheiro, PagamentoCartao, PagamentoCheque) possam acessá-lo diretamente sem necessidade de getter.

5. Classes Abstratas

Questões 4 e 6. Uma classe abstrata serve de modelo para subclasses e não pode ser instanciada com new.

5.1 Regras de uma classe abstrata

- Declarada com abstract class
- Pode ter métodos abstratos (sem corpo) e métodos concretos (com corpo)
- Pode ter construtores, atributos e métodos static normalmente
- NÃO pode ser instanciada diretamente com new
- Métodos abstratos NÃO podem ser private (subclasse não conseguiria sobrescrever)

```
public abstract class Pagamento { // 4.1 — abstract class
    protected double valorHospedagem; // 4.2 — protected

    // Factory Method — cria a subclasse correta
    public Pagamento criarPagamento(String tipo) {
        Pagamento pagamento = null; // 4.3 — inicializar variável local
        switch (tipo) {
            case "Dinheiro":
                pagamento = new PagamentoDinheiro(); // 4.4
                break;
            case "Cartao":
                pagamento = new PagamentoCartao(); // 4.4
                break;
            case "Cheque":
```

```

pagamento = new PagamentoCheque(); // 4.4
break;
}
return pagamento; // 4.5
}
@Override
public String toString() {
return "Valor da hospedagem: R$ " + valorHospedagem; // 4.6
}
public void setValorHospedagem(double valorHospedagem) { // 4.7
this.valorHospedagem = valorHospedagem;
}
}

```

5.2 O que é e o que NÃO é permitido (Questão 6)

Instrucao	Permitida em classe abstrata?
protected abstract void calcular();	Sim — metodo abstrato protected
public abstract void exibir();	Sim — metodo abstrato public
public static final double PI = 3.14;	Sim — constante estatica
private double multiplicar(double n1, double n2){...}	Sim — metodo privado concreto
public Calculo() {}	ATENCAO: o construtor existe, mas a classe nao pode ser instanciada externamente
private abstract int metodo();	NAO — abstract nao pode ser private

6. Herança e super()

Questão 5. Uma subclasse herda atributos e métodos da superclasse usando extends.

6.1 Regras de herança em Java

- Java suporta apenas herança simples entre classes (extends para uma única classe)

- O construtor da subclasse DEVE chamar `super(...)` se a superclasse não tiver construtor sem argumentos
- `super(...)` deve ser a primeira instrução do construtor da subclasse
- Se o atributo herdado for `private`, a subclasse o acessa via `super()` ou `setter/getter`

```
public class PagamentoDinheiro extends Pagamento { // 5.1 — extends
private double valorFornecido;
public PagamentoDinheiro(double valorHospedagem, double valorFornecido) {
super(valorHospedagem); // 5.2 — inicializa atributo da superclasse
this.valorFornecido = valorFornecido; // 5.3
}
}
```

Por que `super(valorHospedagem)` e não `this.valorHospedagem = valor`?

Porque no enunciado da questão 5.2 a variável `valorHospedagem` é `private` na superclasse. Acesso direto seria inválido. O construtor da superclasse (via `super`) é o único caminho correto para inicializá-la.

6.2 Sobrescrita vs Sobrecarga

Criterio	Sobrescrita (Override) vs Sobrecarga (Overload)
Onde ocorre	Override: subclasse Overload: mesma classe
Assinatura	Override: idêntica a do método pai Overload: parâmetros diferentes
Resolução	Override: em runtime (dinâmica) Overload: em compile time (estática)
Anotação	Override: <code>@Override</code> recomendado Overload: nenhuma
Exemplo	Override: <code>Cachorro.emitirSom()</code> redefine <code>Animal.emitirSom()</code> Overload: <code>somar(int,int)</code> e <code>somar(double,double)</code> na mesma classe

7. Semântica de Referências em Java

Questão 7. O tema mais 'pegadinha' da prova — fundamental entender como Java passa referências.

7.1 O que Java passa por valor

Java passa sempre **o valor da referência** (não o objeto em si). Reatribuir o parâmetro local NÃO afeta a variável original.

```
static void manipular(AtribuicaoReferencia r1, AtribuicaoReferencia r2) {  
    r1.setNome("Gregorio"); // altera o OBJETO de p1 — p1.nome = "Gregorio"  
    r2 = r1; // r2 LOCAL passa a apontar para o objeto de r1  
    // p2 ORIGINAL nao e afetado  
    r2.setNome("Ana"); // altera novamente o objeto de p1 — p1.nome = "Ana"  
}  
// main:  
// p1 = "Professor" => apos manipular: p1 = "Ana"  
// p2 = "Estudante" => apos manipular: p2 = "Estudante" (nao mudou)  
// Saida: Ana,Estudante
```

7.2 Diagrama mental para resolver esse tipo de questão

- Desenhe uma seta de cada variável para o objeto na heap
- r1, r2 dentro do método são CÓPIAS das setas de p1, p2
- setNome() via r1 ou r2 atinge o OBJETO apontado — isso reflete em p1 ou p2 se apontarem para o mesmo objeto
- r2 = r1 move a seta CÓPIA de r2 — não move p2

8. Métodos Static vs de Instância

Questão 8. Um erro clássico que o compilador Java rejeita.

8.1 A regra fundamental

```
public class Teste {  
    public static void main(String[] args) {  
        metodo1(); // ERRO: metodo de instancia chamado de contexto static  
    }  
    public void metodo1() { // metodo de INSTANCIA  
        System.out.println("nao compila");  
    }  
}
```

Correções possíveis:

Opcao de correcao	Codigo
-------------------	--------

Tornar metodo1 static	public static void metodo1() { ... }
Criar instancia para chamar	new Teste().metodo1();

Regra geral

Contexto static não tem 'this'. Só pode chamar diretamente outros métodos static da mesma classe. Para chamar método de instância, é preciso de um objeto.

9. Tratamento de Exceções

Tópico da Unidade II — pode aparecer em provas futuras. Base: checked vs unchecked.

Tipo	Característica
Checked (IOException, SQLException...)	Herda de Exception. OBRIGATÓRIO tratar com try-catch ou declarar com throws
Unchecked (RuntimeException...)	NullPointerException, ClassCastException, ArrayIndexOutOfBoundsException. Tratamento opcional
Error	Problemas da JVM (OutOfMemoryError). Não tratar na aplicação

```
// Estrutura completa
try {
    // código que pode lançar exceção
    double r = calcular(10, 0);
} catch (ArithmeticException e) {
    System.out.println("Erro: " + e.getMessage());
} catch (Exception e) { // captura mais ampla por último
    e.printStackTrace();
} finally {
    System.out.println("Executa sempre"); // mesmo com exceção
}

// Exceção customizada
public class ReservaException extends Exception {
```



```
public ReservaException(String msg) { super(msg); }
}
```

10. Cheat Sheet — Tabela Geral da Prova

Conceito	Palavra-chave	Regra principal
Singleton	private static final + private constructor + static getter	Uma instancia, construtor privado, acesso via metodo estatico
Classe abstrata	abstract class	Nao instanciavel; pode ter metodos abstratos e concretos
Metodo abstrato	abstract + sem corpo	Nao pode ser private; obriga subclasse a implementar
Heranca	extends	Java: heranca simples; super() na 1a linha do construtor
Sobrescrita	@Override	Mesma assinatura; resolucao em runtime
Sobrecarga	mesmo nome, params diferentes	Resolucao em compile time; pode ser em construtores
this()	this(args)	Delega para outro construtor da mesma classe; 1a instrucao
super()	super(args)	Chama construtor da superclasse; 1a instrucao
protected	protected	Visivel em subclasses + mesmo pacote
static	static	Pertence a classe; nao pode acessar membros de instancia
Referencia Java	passagem por valor da ref	Reatribuir parametro local nao afeta variavel original
Checked exception	extends Exception	Compilador exige try-catch ou throws na assinatura

11. Armadilhas Mais Comuns

Comparar Strings com ==

== compara referências, não conteúdo. Use sempre .equals() ou
.equalsIgnoreCase() para comparar Strings.

Chamar método de instância de contexto static

main() é static — não existe 'this'. Para chamar método de instância, crie um
objeto: new MinhaClasse().meuMetodo();

Esquecer `super()` quando não há construtor default na superclasse

Se a superclasse não declarou construtor sem argumentos, o compilador não pode inserir `super()` automático. Declare `super(args)` explicitamente na primeira linha.

Confundir sobrescrita com sobrecarga

Override = mesmo nome E mesma assinatura, em subclasse. Overload = mesmo nome, parâmetros diferentes, na mesma classe.

Reatribuir parâmetro de referência esperando alterar o original

`r2 = r1` dentro de um método só muda a cópia local de `r2`. A variável `p2` no chamador continua apontando para o objeto original.

Não inicializar variável local antes de usar

Variáveis locais (dentro de métodos) NÃO são inicializadas automaticamente em Java. Atributos de instância são (`null/0/false`), mas variáveis locais precisam de `=` valor antes do uso.

12. Checklist de Revisão

Antes da prova, confirme que você consegue:

- Escrever uma classe Singleton completa de cabeça
- Criar um array de objetos com contador static para controle de inserção
- Usar `this()` para delegar entre construtores (sobrecarga de construtores)
- Aplicar `this.atributo = parametro` para resolver ambiguidade
- Declarar classe abstrata com métodos abstratos e concretos
- Criar subclasse com `extends` e chamar `super(args)` corretamente
- Explicar por que método static não pode chamar método de instância
- Rastrear o que acontece com referências passadas por parâmetro
- Distinguir sobrescrita (`@Override`) de sobrecarga (mesma classe, params diferentes)
- Explicar a diferença entre `private`, `default`, `protected` e `public`
- Estruturar `try-catch-finally` e criar uma exceção customizada

Boa prova! — Prof. Rodrigo Martins Pagliares — UNIFAL-MG